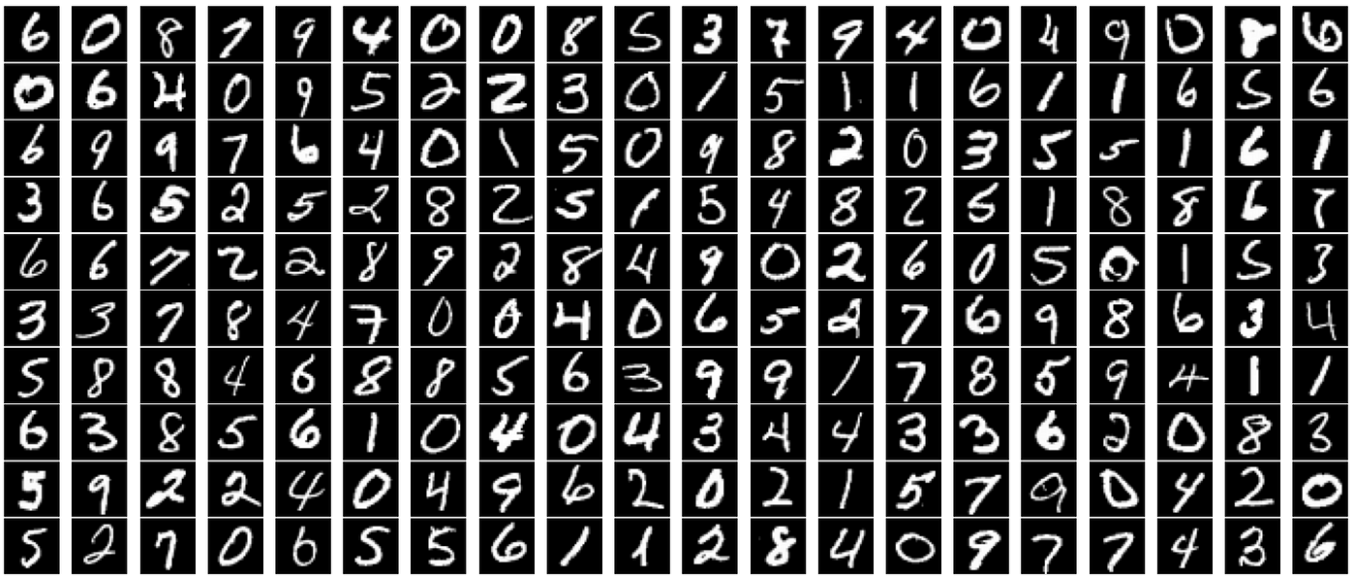
Classificador de Rede Neuronal em RISC-V

Este documento descreve os projetos da disciplina de Introdução à Arquitetura de Computadores 2024/2025 (LEIC-A e LEIC-T).



Histórico de versões

- v1.2: Corrige o tipo do retorno da função classify.
- v1.2: Expande a descrição do 2º projeto com os pormenores necessários para o resolver. Altera os códigos de erro das funções relu e argmax.
- v1.1: Corrige pequena gralha na descrição das excepções do 1º projeto.
- v1.0: Versão inicial do enunciado, que especifica o 1º projeto em pormenor.

1 Introdução

Nestes três projetos, que podem ser vistos como as três fases de um projeto mais longo, irão entrar no mundo da linguagem Assembly RISC-V, explorando na prática conceitos fundamentais como o funcionamento de baixo nível do processador, as convenções de chamada de funções e, até, o projeto de um pequeno processador de 8 bits, dedicado a acelerar algumas das operações de uma rede neuronal.

A vossa missão? Construir, em Assembly, um classificador de dígitos manuscritos com base em algoritmos modernos de Inteligência Artificial! O objetivo é simular uma pequena rede neuronal, utilizando parâmetros de treino (pesos) e dados de entrada previamente definidos, com vista ao reconhecimento de dígitos de 0 a 9 a partir de imagens manuscritas, como as que se mostram na imagem seguinte.

É uma excelente oportunidade para ligarem a teoria à prática e perceberem como algoritmos de inteligência artificial podem ser implementados de forma eficiente, diretamente sobre a interface que o processador expõe ao software – isto é, o Assembly.

Para começarem com o pé direito, recomendamos que assistam ao vídeo abaixo – fornece um bom enquadramento sobre o que irão implementar.

But what is a neural network? | Deep learning chapter 1

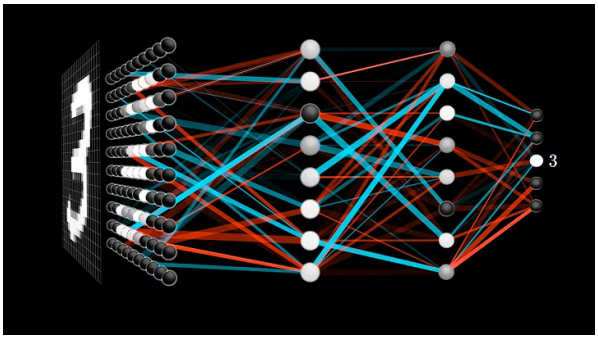


Figura 1: Representação visual da rede neuronal (imagem copiada do vídeo).

Projeto 1: Primeiros módulos

Neste primeiro projeto, cada grupo deve resolver os três problemas de aritmética inteira descritos de seguida.

Cada problema deve ser resolvido como um programa autónomo, pelo que a entrega consistirá em três ficheiros distintos.

1a. Valor absoluto de um inteiro (abs)

Este primeiro passo serve apenas para "aquecer". A função **abs** é uma rotina que calcula o **valor absoluto de um número inteiro** armazenado em memória:

- Carrega o valor apontado por um ponteiro
- Verifica se é negativo
- Se for, armazena o seu simétrico (positivo) na mesma posição de memória.

Argumentos

- `a0 (int *)`: Ponteiro para um único inteiro em memória.

Valores de Retorno

- Nenhum (a função modifica o valor diretamente na memória apontada).

Exceção

- Nenhuma.

1b. Rectified Linear Unit (relu)

A ReLU é uma função de ativação amplamente utilizada em redes neuronais, particularmente em modelos de aprendizagem profunda. Para cada inteiro num vetor de entrada, a função mantém o valor se for positivo; caso contrário, retorna altera-o para zero.

- **Argumentos**
  - `a0` (int *): Ponteiro para o início do vetor de inteiros.
  - `a1` (int): Número de inteiros no vetor.
- **Valores de Retorno**
  - Nenhum (a função modifica os valores diretamente no vetor)
- **Exceção**
  Se a entrada estiver malformada da forma indicada a seguir, deve colocar-se o código de erro apropriado em `a0` e saltar imediatamente para o final do programa com `j exit_with_error`.
  - Código 36 - O comprimento do vetor é menor que 1.

1c. Maior elemento de um vetor (argmax)

Esta função recebe um vetor de inteiros e retorna o **índice do maior elemento**. Se existirem múltiplos elementos com o mesmo valor máximo, deve ser devolvido o menor índice entre eles. Por exemplo, para o vetor de inteiros `[-6, -1, 6, 1]`, o resultado deve ser `2` (índice de `6`). Para o vetor `[6, 1, 6, 1]`, o resultado deve ser `0`, pois o valor `6` aparece pela primeira vez nessa posição.

- **Argumentos**
  - `a0` (int *): Ponteiro para o início do vetor de inteiros.
  - `a1` (int): Número de inteiros no vetor.
- **Valor de Retorno**
  - `a0` (int): Índice do maior elemento do vetor (em caso de empate, o menor índice).
- **Exceção**
  Tal como no exercício anterior, coloque o código de erro correspondente em `a0` e execute `j exit_with_error` para sair do programa.
  - Código 37 – O comprimento do vetor é inferior a 1.

Projeto 2: Compor a Rede Neuronal

Neste projeto iremos implementar mais algumas funções de processamento matricial, reaproveitar a lógica desenvolvida no primeiro projeto (agora organizada em três funções) e, finalmente, integrar todas estas funções para construir a nossa rede neuronal.

A rede neuronal a implementar é do tipo *feedforward* com uma única camada. Todos os pesos são fornecidos. Não é necessário realizar o treino da rede.

Em todas as funções descritas de seguidas, assumimos que os valores inteiros são representados em números com sinal em complemento a dois.

2a. Produto escalar (dotproduct)

Esta função recebe dois vetores de inteiros com o mesmo tamanho e devolve o respetivo produto escalar. Isto é, multiplica os elementos correspondentes entre si e retorna a soma de todos os produtos.

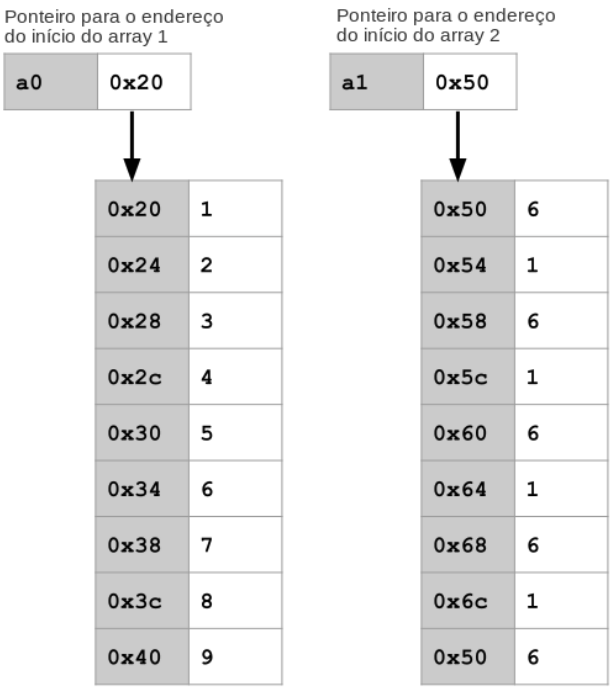


Figura 2: Exemplo da função `DotProduct`.

Por exemplo, se forem passados os dois vetores acima (Figura 2) à função `dotProduct` a função devolverá chegar ao seguinte resultado:

$$(1\times6) + (2\times1) + (3\times6) + (4\times1) + (5\times6) + (6\times1) + (7\times6) + (8\times1) + (9\times6) = \mathbf{170}.$$

Argumentos:

- `a0` (int *): Ponteiro para o início do primeiro vetor.
- `a1` (int *): Ponteiro para o início do segundo vetor.
- `a2` (int): Número de elementos de cada vetor.

Valores de Retorno:

- `a0` (int): Resultado do produto escalar entre os dois vetores.

Exceção:

Se a entrada estiver mal formada da forma abaixo, coloque o código de erro apropriado em `a0` e execute `j exit_with_error` para terminar o programa:

- Código 38: O número de elementos a utilizar é menor que 1.

2b. Multiplicação de matrizes (matmul)

Esta função recebe duas matrizes de inteiros, A (de dimensão $n \times m$) e B (de dimensão $m \times k$), e gera uma matriz de inteiros C (de dimensão $n \times k$). Para calcular o valor da entrada na linha i e coluna j de C, deve calcular-se o produto escalar entre a i-ésima linha de A e a j-ésima coluna de B. Esta função assume que as matrizes estão organizadas na memória por ordem principal de linha (i.e., *row-major order*).

- **Argumentos**
  - `a0` (int *): Ponteiro para o início da matriz A.
  - `a1` (int *): Ponteiro para o início da matriz B.
  - `a2` (int): Número de linhas (altura) da matriz A.
  - `a3` (int): Número de colunas (largura) da matriz A.
  - `a4` (int): Número de linhas (altura) da matriz B.
  - `a5` (int): Número de colunas (largura) da matriz B.
  - `a6` (int *): Ponteiro para o início da matriz C, que se assume ter a dimensão esperada.

- **Valores de Retorno**
  - Nenhum
- **Exceções**
  Se a entrada estiver mal formada de uma das formas seguintes, coloque o código de erro apropriado em `a0` e execute `j exit_with_error` para terminar o programa.
  - Código 39 - A altura ou a largura de qualquer uma das matrizes é inferior a 1.
  - Código 40 - O número de colunas da matriz A não é igual ao número de linhas da matriz B.

Nota: valorizaremos soluções em que a função *matmul* é composta por chamadas à função *dotproduct*. Para cumprir esse requisito, podem adicionar à *dotproduct* os argumentos que julguem necessários (mas sem modificar os argumentos já definidos neste enunciado).

2c. Leitura de ficheiros (read_file)

A função read_file abre um ficheiro cujo conteúdo é um conjunto de inteiros, e carrega o seu conteúdo para o *buffer* passado por argumento. Esta função deve usar as chamadas sistema fornecidas pelo Ripes para acesso a ficheiros.

Antes de implementar esta função, deve resolver o guião laboratorial que será disponibilizado sobre este tema.

Argumentos:

- `a0` (int *): Ponteiro para a string do nome (*path name)* do ficheiro a ser lido.
- `a1` (int *): Ponteiro com o buffer que receberá os dados do ficheiro.
- `a2` (int): Número de bytes a ser lido.

Exceções:

- Código 41 - Alguma chamada sistema (*open* ou *read*) devolveu erro.
- Código 42 - O número de bytes passado por argumento é menor que 1.

2d. Compor a rede neuronal

Lembrem-se: estamos a construir uma rede neuronal simples, passo a passo. Nesta tarefa, irão utilizar as funções desenvolvidas nas tarefas anteriores para completar a rotina de classificação, denominada classify.

Argumentos e retorno

Argumentos:

- `a0 (string *)`: Endereço de *string* com o *path name* do ficheiro com a matriz de pesos m0.
- `a1 (string *)`: Endereço de *string* com o *path name* do ficheiro com a matriz de pesos m1.
- `a2 (string *)`: Endereço de *string* com o *path name* do ficheiro com a matriz de entrada, *input* (i.e., a imagem a classificar).

Retorno:

- `a0 (int)`: Inteiro com o valor do algarismo classificado.

Dados

A rotina classify usa as seguintes variáveis globais, que se assume que são alocadas estaticamente (na secção *.data* do programa):

- **m0**, **m1**, **input**, **h**, **o**: as matrizes bidimensionais usadas pelo algoritmo
- w_m0, w_m1, w_input, w_h, w_o: largura de cada matriz
- h_m0, h_m1, h_input, h_h, h_o: altura de cada matriz

Algoritmo

De seguida descrevemos o pseudo-código da rotina classify, que está ilustrado nas figuras na página seguinte:

1. Carrega as matrizes **m0**, **m1** e **input** a partir dos *path names* passados por argumento (chamando a função **read_file**).
2. Computar **h = matmul(m0, input)**.
3. Executar **relu(h)**. Lembre-se que a função *relu* é executada diretamente sobre a matriz passada por referência.
4. Computar **o = matmul(m1, h)**.
5. Executar **argmax(o)**, sendo que o valor retornado corresponde ao valor classificado.
6. Imprima o valor na consola (usando a *ecall PrintInt*).
7. Retorne o valor em a0.

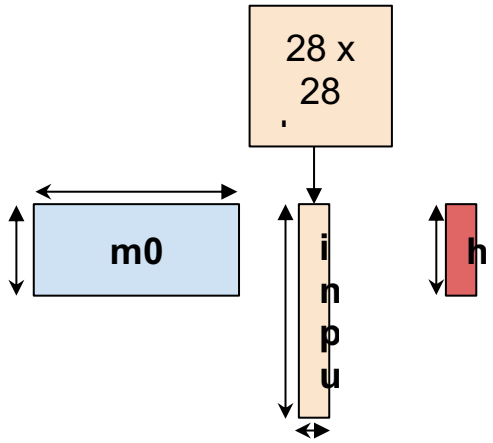


Figura 3: Representação visual da operação h = matmul (m0, input).

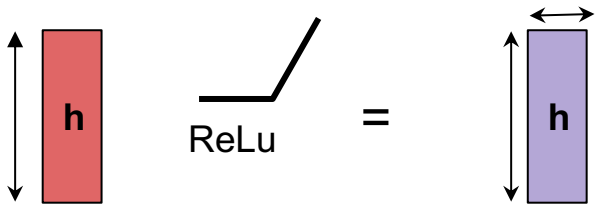


Figura 6: Representação visual da operação h = relu (h).

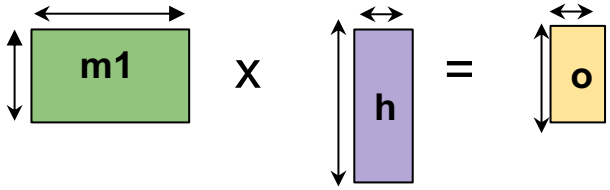


Figura 5: Representação visual da operação o = m1 x h.

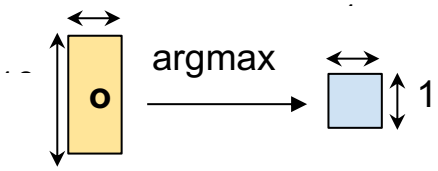


Figura 6: Representação visual de resultado = argmax(o).

Projeto 3: Uma máquina de estados para acelerar operações da rede neuronal

Neste projeto, construirão um circuito sequencial, mais precisamente um processador de 8 bits que será especializado em algumas operações muito importantes para a vossa rede neuronal.

Esse processador será uma versão muito simplificada daquilo que hoje em dia se chama uma *tensor processing unit (TPU)*. Ela re-implementará a lógica das funções `abs` e `relu` do 1º projeto. No entanto, como essa implementação é feita em *hardware*, será mais otimizada. Portanto, este processador será aquilo que designamos um *acelerador*.

Posteriormente publicaremos uma extensão a este enunciado com pormenores sobre o 3º projeto.

4 Considerações práticas

O projeto deve ser desenvolvido no simulador **Ripes** (1º e 2º projetos) e ferramenta **Logisim** (3º projeto), que serão apresentados nas aulas laboratoriais. O processador simulado em Ripes deve ser um **RISC-V de 32 bits**, de acordo com a arquitetura estudada nas aulas teóricas e laboratoriais.

No caso dos 1º e 2º projetos, forneceremos os ficheiros de código iniciais (através do Fénix). As soluções devem **obrigatoriamente** ser construídas sobre esses ficheiros, respeitando a estrutura fornecida inicialmente.

5 Entregas e avaliação

Os projetos são submetidos no fénix, com os seguintes prazos:

- Projeto 1: 23h59 de 9/maio
- Projeto 2: 23h59 de 30/maio
- Projeto 3: 23h59 de 16/junho

Nota mínima de 9,0.

O peso de cada projeto na nota final pode ser consultado nos métodos de avaliação no fenix.

Embora o projeto seja desenvolvido em grupos de 3 estudantes, após as 3 submissões haverá uma discussão cujo objetivo é aferir se cada membro do grupo contribuiu e domina as soluções que foram submetidas pelo grupo. A discussão inclui um momento em que se pede a cada membro do grupo que faça modificações menores às soluções submetidas pelo grupo (no fundo, um pequeno teste prático).

Usaremos software de deteção de cópias. Casos de plágio seguem os procedimentos do regulamento de avaliação do IST.